

Práctica 3: Primeros módulos Odoo

Andrea Sofía Pais Dos Santos

December 2, 2025

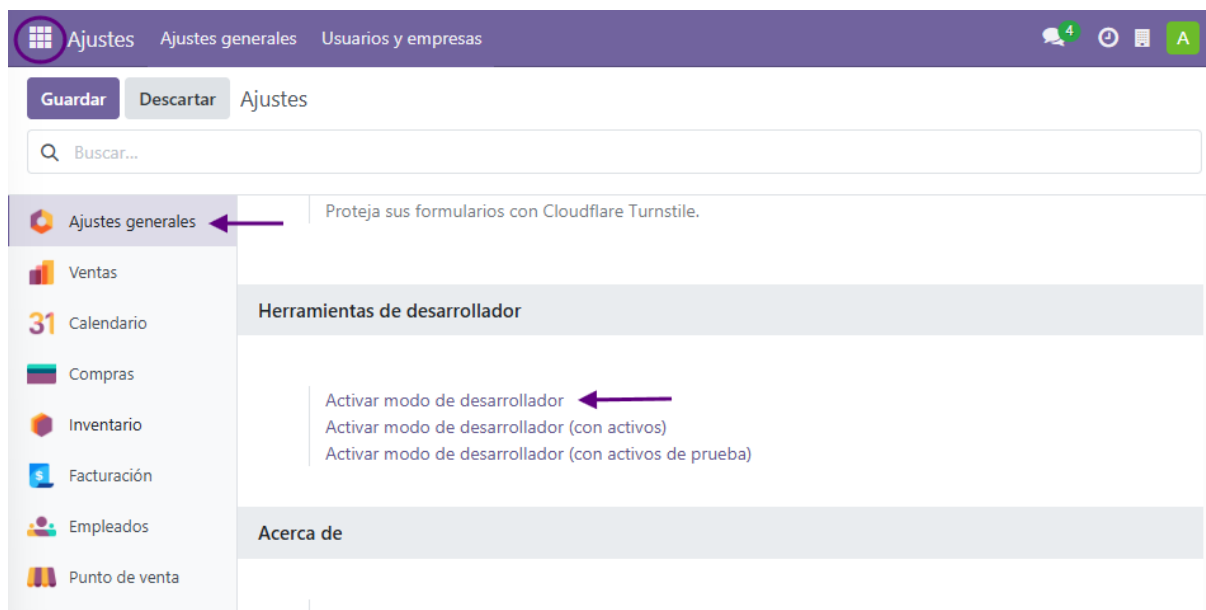
1 Índice

2. Introducción	página 3
3. Implementación de módulo básico	página 4
• Instalación del módulo "hola mundo"	
4. Implementación del primer módulo	página 5
• Instalación del módulo "lista de tareas"	
• Problemas encontrados y soluciones	
5. Modificación del primer módulo	página 10
• Modificación del módulo "lista de tareas"	
6. Conclusiones	página 12

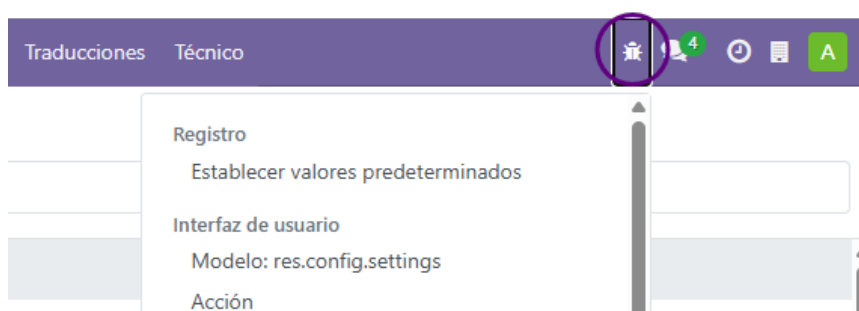
2 Introducción

El objetivo de la práctica es mediante la creación y la implementación de módulos básicos, familiarizarse con el entorno de desarrollo de Odoo. Primero lo que vamos a tener que hacer es activar el Modo Desarrollador, para ellos accedemos al enlace [Cambio a modo desarrollador](#).

Para activar el modo desarrollador, nos situamos en "Ajustes Generales", bajamos hasta encontrar "Herramientas de desarrollador" y activamos la primera opción.



Al activar el modo desarrollo nos aparece un icono de un "insecto" con un submenú que contiene herramientas para entender o editar información técnica.



3 Implementación de módulo básico

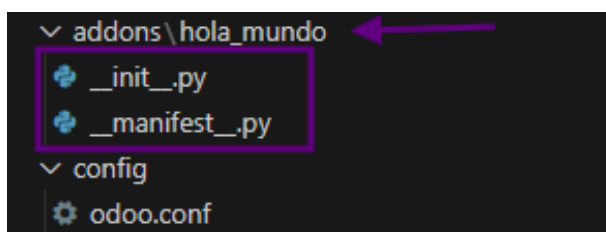
3.1 Instalación del módulo "Hola mundo"

- Creamos una carpeta para el módulo "hola mundo" en la carpeta addons.

```
mkdir addons/hola_mundo
```

- Creamos los archivos vacíos siguientes:

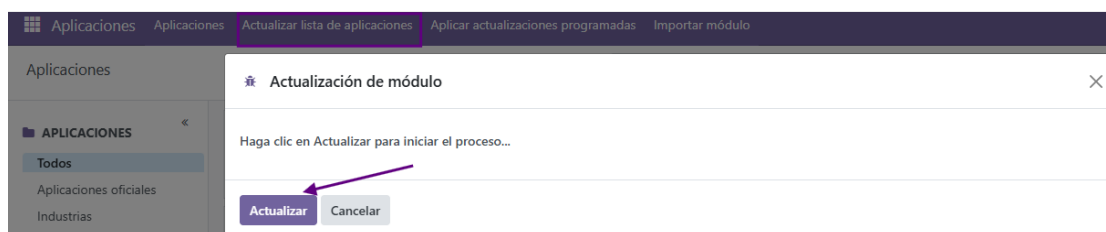
```
__init__.py  
__manifest__.py
```



- Al ser un módulo sencillo en primer archivo "init" puede estar vacío, mientras que al archivo "manifest" le vamos a añadir el siguiente ejemplo:

```
{  
    'name': 'Hola Mundo',  
    'summary': 'Mi primer módulo Odoo',  
    'description': 'Módulo básico de prueba para la práctica 3',  
    'author': 'Andrea Sofía Pais Dos Santos',  
    'version': '1.0',  
    'depends': ['base'],  
    'application': True,  
    'category': 'Tools',  
}
```

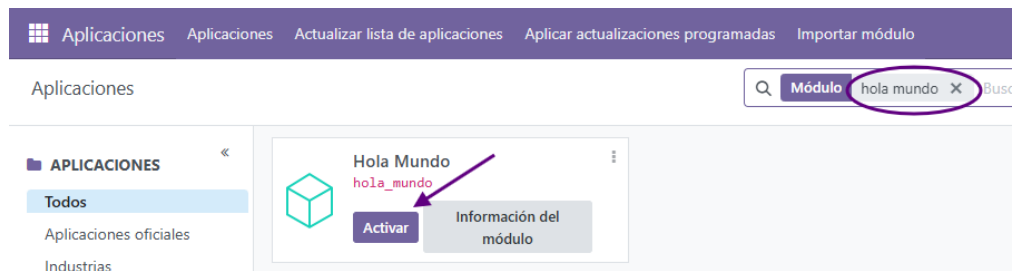
- En el entorno de desarrollo actualizamos la "lista de aplicaciones", buscamos el módulo "hola mundo".



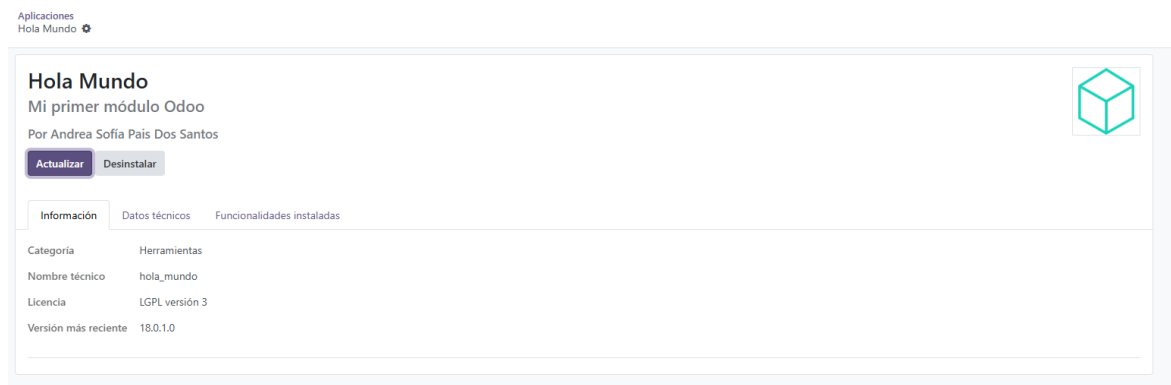
- Si haciendo lo anterior sigue sin salir el módulo, ejecutamos y volvemos a actualizar la lista de aplicaciones.

```
docker-compose restart web
```

- Buscamos el módulo "Hola Mundo" y lo activamos.



- En aplicaciones instaladas encontraremos el módulo "Hola mundo" con su respectiva información.



4 Implementación del primer módulo

4.1 Instalación del módulo "lista de tareas"

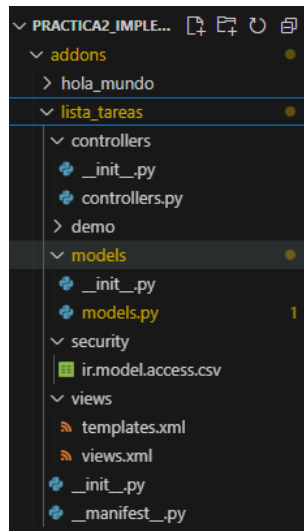
- Creamos una carpeta para el módulo "lista de tareas"

```
mkdir addons/lista_tareas
```

- Para ayudarnos a crear el primer módulo vamos a usar la siguiente guía: [Primer módulo](#), en el que vamos a usar "una plantilla de estructura de módulo" para realizarlo manualmente en mi caso.

Si no desea instalar Odoo en su computadora, puede [descargar esta plantilla de estructura de módulo](#) en la cual se reemplaza cada ocurrencia de *my_module* con el nombre de su preferencia.

- Al descargar la plantilla en la carpeta que creamos nos genera la siguiente estructura de carpetas y archivos que modificaremos con la información que nos interese.



Ahora vamos a rellenar los archivos con el código necesario para crear el módulo.

- Vamos a empezar con los archivos "init", el primero "`addons/listas_tareas/__init__.py`", le podemos estas importaciones:

```
from . import controllers
from . import models
```

- Para el archivo "`addons/listas_tareas/models/__init__.py`" lo tenemos que rellenar con lo siguiente:

```
from . import models
```

- Para el archivo "`addons/listas_tareas/controllers/__init__.py`" le añadimos esto:

```
from . import controllers
```

- Ahora vamos con los archivos que tienen más contenido, empezamos con el archivo "`addons/listas_tareas/__manifest__.py`" le añadimos estos datos:

```
# -*- coding: utf-8 -*-
{
    'name': "Lista de tareas",
    'summary': """"Sencilla Lista de tareas""",
    'description': """"Sencilla lista para crear nuevo módulo""",
    'author': "Andrea Sofía Pais Dos Santos",
    'application': True,
    'category': 'Productivity',
    'version': '0.1',
    'depends': ['base'],
    'data': [
        'security/ir.model.access.csv',
        'views/views.xml',
    ],
}
```

- Ahora dentro del archivo "[addons/lista_tareas/controllers/controllers.py](#)" es un archivo que puede quedar vacío como en este caso.
- Dentro del archivo "[addons/lista_tareas/demo/demo.xml](#)" es un archivo que puede quedar vacío como en este caso.
- Ahora dentro del archivo "[addons/lista_tareas/models/models.py](#)" tenemos que añadir:

```
from odoo import models, fields, api
#Definimos el modelo de datos
class lista_tareas(models.Model):
    #Nombre y descripcion del modelo de datos
    _name = 'lista_tareas.lista_tareas'
    _description = 'lista_tareas.lista_tareas'
    #Elementos de cada fila del modelo de datos
    #Los tipos de datos a usar en el ORM son
    tarea = fields.Char(string="Tarea")
    prioridad = fields.Integer(string="Prioridad")
    urgente = fields.Boolean(compute="_value_urgente", store=True)
    realizada = fields.Boolean(string="Realizada")
    @api.depends('prioridad')
    def _value_urgente(self):
        #Para cada registro
        for record in self:
            #Si la prioridad es mayor que 10, se considera
            urgente, en otro caso no lo es
            if record.prioridad>10:
                record.urgente = True
            else:
                record.urgente = False
```

- Ahora dentro del archivo "[addons/lista_tareas/security/ir.model.access.csv](#)" tenemos que añadir:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,
perm_unlink
access_lista_tareas_user,lista.tareas.user,
model_lista_tareas_lista_tareas,base.group_user,1,1,1,1
```

- Ahora dentro del archivo "[addons/lista_tareas/views/view.xml](#)" tenemos que añadir:

```
<odoo>
<data>
<!-- explicit list view definition -->
<!-- Definimos como es la vista explicita de la listas-->
<record model="ir.ui.view" id="lista_tareas.list">
    <field name="name">lista_tareas list</field>
    <field name="model">lista_tareas.lista_tareas</field>
    <field name="arch" type="xml">
        <list>
            <field name="tarea"/>
```

```

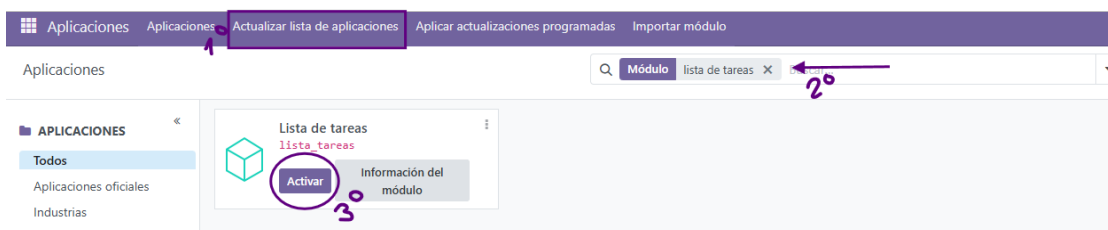
        <field name="prioridad"/>
        <field name="urgente"/>
        <field name="realizada"/>
    </list>
</field>
</record>
<!-- actions opening views on models -->
<!-- Acciones al abrir las vistas en los modelos-->
<record model="ir.actions.act_window" id="lista_tareas.action_window">
    <field name="name">Listado de tareas pendientes</field>
    <field name="res_model">lista_tareas.lista_tareas</field>
    <field name="view_mode">list,form</field>
</record>
<!-- Top menu item -->
<menuitem name="Listado de tareas" id="lista_tareas.menu_root"/>
<!-- menu categories -->
<menuitem name="Opciones Lista Tareas" id="lista_tareas.menu_1"
parent="lista_tareas.menu_root"/>
<!-- actions -->
<menuitem name="Mostrar lista" id="lista_tareas.menu_1_list"
parent="lista_tareas.menu_1" action="lista_tareas.action_window"/>
</data>
</odoo>

```

- Después de tener todos los archivos vamos a resetear, ejecutando lo siguiente:

```
docker-compose restart web
```

- Actualizamos la lista de aplicaciones como hicimos anteriormente y buscamos el módulo "lista de tareas" creado:

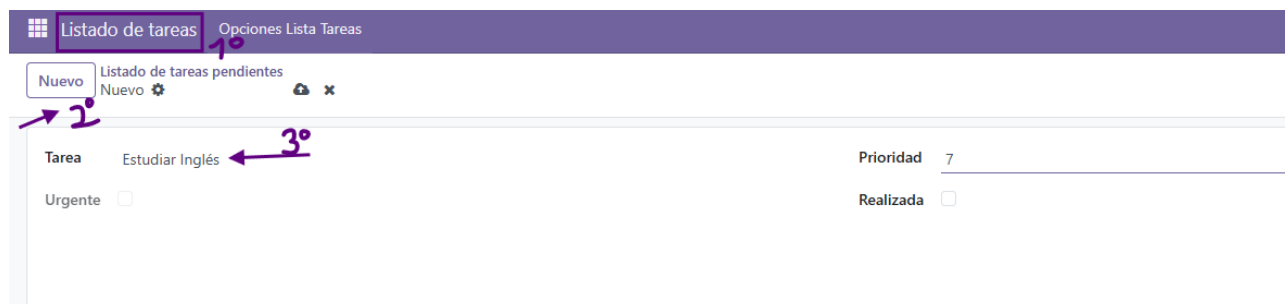


4.2 Problemas encontrados y soluciones

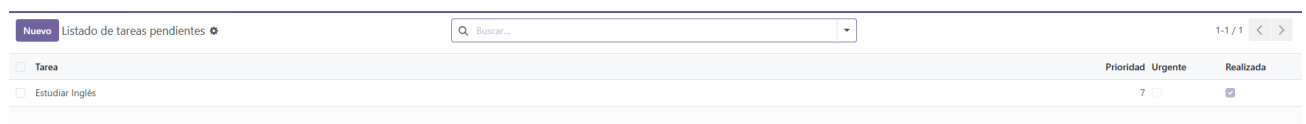
Al activar el módulo de "lista de tareas" me sale un error que no me permite activar el módulo, investigué y parece ser que había que cambiar "tree" (formaba parte de la terminología antigua) por "list" (terminología nueva). "Tree" era para Odoo 16, ahora para Odoo 18 se usa "List".



Ya al cambiar eso me permite activar el módulo "lista de tareas". Ahora ya podemos crear nuestras tareas, aquí una de ejemplo:



Se pueden modificar en cualquier momento, añadir más, borrarlas, etc.



5 Modificación del primer módulo

5.1 Modificación del módulo "lista de tareas"

Para este módulo como modificación extra se decidió agregar un apartado de un listado de categorías para clasificar las tareas y darle un poco más de estética a la interfaz al crear una nueva tarea.

Primero modificamos los archivos, en este caso tenemos que modificar: [addons/lista_tareas/models/models.py](#) y [addons/lista_tareas/views/views.xml](#). En el primero vamos a añadir las categorías que nos interesa:

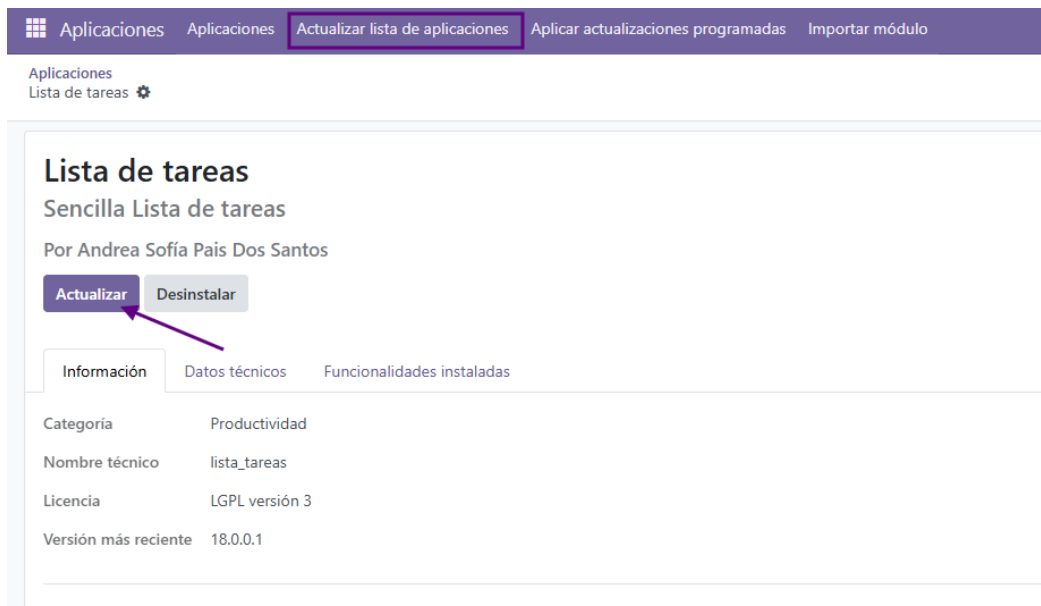
```
categoria = fields.Selection([
    ("estudio", "Estudio"),
    ("trabajo", "Trabajo"),
    ("ocio", "Ocio"),
    ("citas", "Citas")
], string="Categoría", default="cita")
```

Hemos añadido el nombre de las categorías que nos interesa y establecemos por defecto la categoría citas (por ejemplo para médico, entrevistas, eventos, etc).

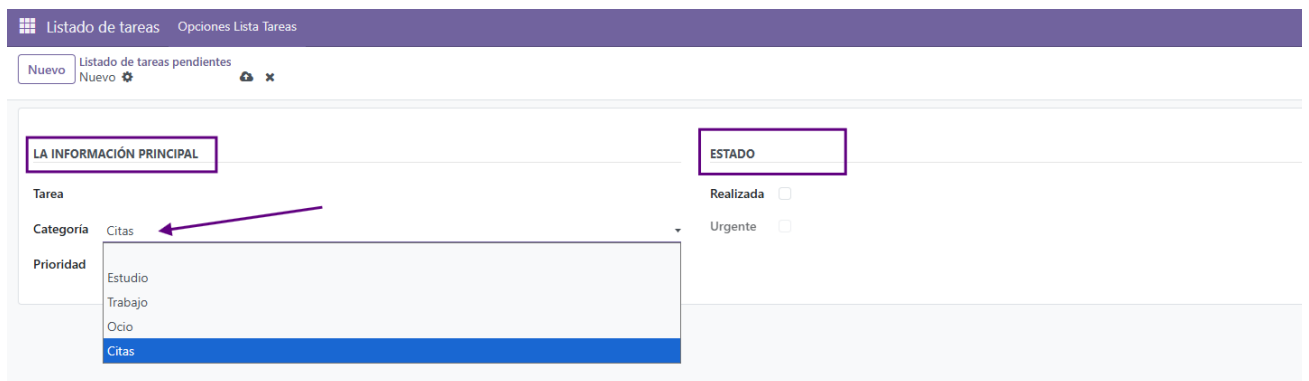
Luego tenemos que añadir una modificación de la vista añadiendo la nueva funcionalidad que acabamos de crear:

```
<record model="ir.ui.view" id="lista_tareas_form">
  <field name="name">lista_tareas_form</field>
  <field name="model">lista_tareas.lista_tareas</field>
  <field name="arch" type="xml">
    <form>
      <sheet>
        <group>
          <!-- va a ser el título de este grupo -->
          <group string="La información principal">
            <field name="tarea" required="1"/>
            <field name="categoria"/>
            <field name="prioridad"/>
          </group>
          <!-- va a ser el título de este grupo -->
          <group string="Estado">
            <field name="realizada"/>
            <field name="urgente" readonly="1"/>
          </group>
        </group>
      </sheet>
    </form>
  </field>
</record>
```

Luego de tener el código ya cambiado, nos vamos al entorno Odoo y volvemos a actualizar la lista de aplicaciones. Buscamos el módulo ya instalado "lista de tareas" y lo actualizamos.



Tras actualizar, nos encontramos con algunos cambios al crear una nueva tarea, nos aparece un desplegable con las opciones de categoría que creamos, y nos separa los bloques con un título descriptivo.



6 Conclusiones

En esta práctica pude entender mejor cómo funciona Odoo por dentro y cómo se crean los módulos desde cero. Empecé con algo simple como el módulo “Hola Mundo” y luego pasé al de “Lista de tareas”, donde ya pude aplicar más cosas y ver cómo se conectan los modelos, vistas y datos.

También me encontré con algunos errores que me ayudaron a entender mejor cómo se estructura todo y cómo solucionarlos. En general, fue una práctica útil para conocer mejor el entorno de desarrollo de Odoo y tener una base para seguir creando módulos más completos más adelante.